

Near-real-time Solutions for Online String Problems

Dominik Köppl

Gregory Kucherov

26th Korea-Japan Joint Workshop on Algorithms and Computation (WAAC 2026)

Abstract

Based on the Breslauer–Italiano online suffix tree construction algorithm (2013) with double logarithmic worst-case guarantees on the update time per letter, we develop near-real-time algorithms for several classical problems on strings, including the computation of the longest repeating suffix array and the Lempel–Ziv 77 factorization in an online manner. Our solutions improve over the best known running times for these problems in terms of the worst-case time per letter, for which we achieve a *poly-log-logarithmic* time complexity, within a linear space. Best known results for these problems require a *poly-logarithmic* time complexity per letter or only provide amortized complexity bounds.

Keywords: String algorithms, online algorithms, suffix tree, real-time computation, Lempel–Ziv factorization

1 Introduction

An important class of string problems concerns *online* processing of large strings when the string is read letter by letter from left to right, while maintaining the desired information about the string read so far. This type of solution is often required in practice. Unfortunately, most online algorithms for strings provide only *amortized* time guarantees per letter, i.e., the average time spent to process each letter is bounded by a function of the input size, but some read letters may take much longer to process. This can be problematic in real-time applications, where it is important to have predictable processing times for each letter. In this paper, we focus on online algorithms with *worst-case* time guarantees per letter. While literature gives real-time algorithms for LZ78-like factorizations [31] with a static dictionary [14] for constant alphabets, palindrome recognition and pattern matching [11], for many classical problems on strings, online algorithms with efficient worst-case time guarantees per letter are not known. In fact, a majority of string problems can be only solved efficiently if we maintain a full-text indexing data structure of the string read so far. One of the most fundamental such data structures is the suffix tree [28]. The first online construction of the suffix tree is due to Ukkonen [27], which builds a suffix tree online in time $O(n)$ for a constant-sized alphabet. However Ukkonen’s algorithm does not provide any worst-case time guarantee on processing an individual letter, other than a trivial $O(n)$ bound. For certain inputs, this bound is actually tight: Given a string SSc of length n , Ukkonen’s algorithm maintains an active node of string depth at least $|S|$ after having parsed the prefix SS , but then needs to add branches for all suffixes of S in case c is a new letter, and thus creates $\Theta(n)$ nodes just for updating the suffix tree for the last letter. (Spending $\Theta(n)$ for the last letter may be a common phenomenon in practice since we usually append a unique delimiter letter $\$$ at the end of the input string.) To achieve worst-case time guarantees, a line of research is based on the idea to maintain a suffix tree (or similar data structure) for the *inverted* input string. The rationale for this is that appending a new letter to the input string amounts to introducing only one new suffix in the inverted string, which causes less updates to the data structure compared to Ukkonen’s algorithm. The starting point of this line of research is Weiner’s algorithm [28], historically the first linear-time algorithm for suffix tree construction for constant-sized alphabets. Weiner’s algorithm processes the string right-to-left by introducing a new suffix at each step, and is therefore suitable for this

task. In its original version, Weiner’s algorithm does not provide a worst-case time guarantee on processing a single letter. However, in the following Section 2.1, we summarize modifications that accomplish this. These modifications result in double logarithmic worst-case times per letter, a time complexity referred to as *near-real-time* [4].

2 Preliminaries

We assume the *word RAM* with word size $w \geq \log n$ bits, where n is the length of the input string. Let Σ denote an integer alphabet of size $\sigma = |\Sigma| = n^{O(1)}$. An element of Σ^* is called a *string*. Given a string $S \in \Sigma^*$, we denote its length with $|S|$, its i -th letter with $S[i]$ for $i \in [1..|S|]$. Further, we write $S[i..j] = S[i] \cdots S[j]$. We write $\overleftarrow{S}$ to denote the *inverted* string of S , i.e., $\overleftarrow{S} = S[|S|]S[|S| - 1] \cdots S[1]$. The *suffix tree* $\text{ST}(S)$ of a string S is a compacted trie representing all suffixes of S .

In what follows, we fix a string $T[1..n]$ over Σ as the input string to be processed online. Here, online means that the algorithm reads T letter by letter and maintains the result for the processed portion of the string. All presented algorithms in this paper use $O(n)$ words of space, unless otherwise stated. Note that this means that the space is proportional to the size of the currently processed string (as opposed to the size of the entire string).

2.1 Breslauer–Italiano modification of Weiner’s algorithm

Weiner’s algorithm computes the suffix tree $\text{ST}(T)$ of a string $T[1..n]$ by processing it in backward direction and inserting suffixes from the shortest $T[n..]$ to the longest $T[1..]$. The algorithm augments suffix tree nodes with additional pointers called *Weiner links* (W-links for short): a *W-link* labeled with letter c from a node α (denoted by $W_c(\alpha)$) points to the locus of string $c \cdot \text{label}(\alpha)$, where $\text{label}(\alpha)$ is the string spelled out by the path from the root to α . $W_c(\alpha)$ is defined only if $c \cdot \text{label}(\alpha)$ is a substring of the current string. If the locus of $c \cdot \text{label}(\alpha)$ is a node, the W-link is called *hard*, otherwise it is called *soft*. In the Breslauer–Italiano modification of Weiner’s algorithm, a soft W-link is represented by a pointer to the closest descendant node of the locus of $c \cdot \text{label}(\alpha)$.

A round of Weiner’s algorithm corresponds to the processing of a newly read letter. At round i , the algorithm inserts the new suffix to the suffix tree, solving the following problem.

SUFFIXUPDATE

Input: suffix tree $\text{ST}(T[i + 1..])$ and letter $T[i]$.

Output: $\text{ST}(T[i..])$.

To this end, it locates the *insertion point* where a new leaf should be attached. The insertion point is the locus of the longest prefix of $T[i..]$ already present as substring in $T[i + 1..]$, it can be either an existing node or a locus on an existing edge. Computing the insertion point is the key operation of Weiner’s algorithm, which we formulate as follows.

INSERTIONPOINT

Input: suffix tree $\text{ST}(T[i + 1..])$ and letter $T[i]$.

Output: the locus of the longest prefix of $T[i..]$ appearing in $T[i + 1..]$.

Solving INSERTIONPOINT or the whole update process is sometimes called the *suffix tree oracle* [9, 26, 7].

Algorithmic steps of one round We break down one round of Weiner’s solving SUFFIXUPDATE into steps, and explain each step in the context of Breslauer and Italiano’s algorithm [4]. Let $c = T[i]$.

1. Starting from the leaf λ corresponding to suffix $T[i + 1..]$, find its lowest ancestor α such that $W_c(\alpha)$ is defined.

2. Let $\beta = W_c(\alpha)$. If $W_c(\alpha)$ is hard, then β is the insertion point. Otherwise, split the parent edge of β and create a new node γ . Copy all W-links from node β to γ which all become soft.
3. Create a new leaf λ' for suffix $T[i..]$ as a child of the insertion point.
4. Create a hard W-link $W_c(\lambda) = \lambda'$ and a soft W-link $W_c(q) = \lambda'$ for each node q on the path between λ (excluded) up to α (excluded).
5. Create a hard W-link $W_c(\alpha) = \gamma$ and a soft W-link $W_c(q) = \gamma$ for each node q on the path between α (excluded) up to the first node δ for which $W_c(\delta) \neq \beta$ (excluded).

To analyze the time complexity of the above steps, we need the following two properties of Weiner's algorithm, which are known results but restated for the sake of completeness.

Lemma 2.1. The height of $ST(T[i..n])$ is the height of $ST(T[i+1..n])$ increased by at most one.

Lemma 2.2. Assume an internal node ζ in $ST(T[i..])$ is the locus of substring cX , i.e., $\text{label}(\zeta) = cX$. Then X is the locus of a node δ in $ST(T[i+1..])$, in particular, $W_c(\delta) = \zeta$ in $ST(T[i..])$ and the depth of ζ is at most the depth of δ plus one.

Time complexity analysis Implemented naively, Step 1 can visit up to $O(n)$ nodes, Step 2 may have to copy $O(\sigma)$ W-links, and Steps 4–5 can update the W-links of $O(n)$ nodes in the worst case. However, assuming a constant-size alphabet, the total time complexity over all n rounds can be shown to be $O(n)$ as follows. Step 1 traverses $(\text{depth}(\lambda_{i+1}) - \text{depth}(\delta))$ nodes, where λ_i stands for the leaf labeling the entire string $T[i..]$. By Lemma 2.2, we have $\text{depth}(\zeta) \leq \text{depth}(\delta) + 1$, and then $\text{depth}(\lambda_{i+1}) - \text{depth}(\delta) \leq \text{depth}(\lambda_{i+1}) - \text{depth}(\zeta) + 1 = \text{depth}(\lambda_{i+1}) - \text{depth}(\lambda_i) + 3$. Summing up over all rounds, Step 1 visits $O(n)$ nodes altogether. By a similar argument, Steps 4–5 update up to $O(n)$ W-links as well. Finally, Step 2 needs $O(n)$ time in total since there are at most n hard W-links in the entire suffix tree and there are $O(1)$ W-links to copy at each round.

To achieve a nontrivial worst-case time bound per letter, we need to efficiently implement Steps 1 and Steps 4–5. Breslauer and Italiano [4] proposed to implement Step 1 by maintaining, for each alphabet letter a , the (dynamic) Euler tour list of the current suffix tree where nodes are *marked* in color a according to W_a -links defined in them. Then, the ancestor node α is retrieved using two data structures: one supporting queries for the *previous marked element* in a dynamic list, applied to the Euler tour, and the other supporting *dynamic lowest common ancestor* applied to the suffix tree itself. The first is solved using the data structure of Dietz and Raman [8] and the second using the data structure of Cole and Hariharan [6]. Altogether, this results in $O(\log \log n)$ time. We note in passing that for a constant-size alphabet, dynamic lowest common ancestor queries can be easily avoided, therefore only colored predecessor queries are critical.

To efficiently implement updates of Steps 4–5, the authors propose to distribute this work over multiple rounds of the algorithm (i.e. multiple letters of the input string) by updating only a constant number at each round. (For this reason, the algorithm is called *quasi-real time*.) However, the nodes are updated in the order of increasing depth which guarantees no interference with the ongoing process of tree updating. As a result, Breslauer and Italiano modification spends $O(\log \log n)$ worst-case time on each letter.

The central observation for this paper is that this modification, as well as the improvements in the following subsection, maintain the suffix tree within this worst case-time per letter while keeping nearly complete functionality.

Observation 2.3. The suffix tree updated by the Breslauer–Italiano algorithm is fully functional except that some W-links may be missing temporarily (due to the deferred updates in Steps 4–5).

2.2 Improvements of Breslauer–Italiano algorithm

Altogether, the Breslauer–Italiano algorithm achieves worst-case $O(\sigma \log \log n)$ time per read letter, where σ is the alphabet size. In what follows, we study alternative worst-case time complexities per read letter for the online suffix tree construction problem, which we denote by t_{SU} . For $\sigma = O(1)$ this results in $O(\log \log n)$ time per read letter. This technique was the first to achieve a double logarithmic worst-case bound, as only an $O(\log n)$ bound had been known previously [1].

An obvious weakness of the Breslauer–Italiano approach is the linear dependency on the alphabet size. Subsequent works proposed improvements on this point. Kopelowitz [15] proposed a randomized algorithm for maintaining the suffix tree online in $O(\log \log n + \log \log \sigma)$ worst-case *expected* time per letter. Kucherov and Nekrich [16] generalize the $O(\log \log n)$ bound of [4] to log-sized alphabets, more precisely to $\sigma = O(\log^{1/4} n)$, using the *generalized van Emde Boas data structure* of Giora and Kaplan [12]. An interesting improvement proposed in [16] is that weak W-links are not maintained at all but are computed on the fly in the lazy fashion. This greatly simplifies the steps of the Breslauer–Italiano algorithm: Steps 2, 4 and 5 become constant time and only Step 1 remains to be implemented. The latter is essentially reduced to answering the *colored predecessor queries* on a dynamic colored list, for which the authors borrow an $O(\log \log n)$ solution of [12] for up to $O(\log^{1/4} n)$ colors. Mortensen [17] proposes an $O(\log^2 \log n)$ solution for the dynamic colored predecessor problem for any number of colors, which can be plugged in as an alternative to compute INSERTIONPOINT.

On the other hand, Fischer and Gawrychowski [9] obtain $O(\log \log n + \frac{\log^2 \log \sigma}{\log \log \log \sigma})$ time using a sophisticated technique of *Wexponential search trees*. They store only some of the weak W-links and show how to implement INSERTIONPOINT within this time bound. Steps 4 and 5 can be deamortized as in the Breslauer–Italiano algorithm.

Note finally that the idea of maintaining online an indexing data structure for the inverted string has been applied to indexes based on the Burrows–Wheeler transform (BWT) [5] as well. Similar to the suffix tree, it is more convenient to maintain a BWT-index for the inverted string, by adding only one suffix at each round [22, 19]. Updating the BWT-index boils down to maintaining a dynamic string that supports access, rank, and select queries, for which logarithmic solutions exist [18], which, however, are optimal [10]. Thus, there is no great hope for improving the time bounds for online BWT-index maintenance beyond logarithmic time per letter. In what follows, we focus on a line of research that applies this online BWT-index construction for computing the LRS array, and show that we can obtain better time bounds by relying on the suffix tree construction instead.

3 Longest Repeating Suffix Array

From now on, we will be considering the online setting where the input string T is provided letter by letter from left to right, and we will be maintaining a suffix tree of the *inverted* string $\bar{T}$, using a variant of Weiner’s algorithm. Thus, a *suffix* of a current string will become an *inverted prefix* of the actually indexed string, etc. In our presentation, we will consider both the string and its inverted image and alternate between the two depending on the context which should be clear to the reader.

The *longest repeating suffix array* (LRS) of a string $T[1..n]$ is an array $\text{LRS}[1..n]$ such that for each position $i \in [1..n]$, $\text{LRS}[i]$ is the length of the longest suffix of $T[1..i]$ that occurs at least twice in $T[1..i]$. In other words, $\text{LRS}[i]$ is the length of the longest suffix of $T[1..i]$ occurring also in $T[1..i-1]$.

Okanohara and Sadakane [20] were the first to study computing this array online, for which they obtained $O(\log^3 n)$ worst-case time per letter. Prezza and Rosone [23] obtained $O(\log^2 n)$ *amortized* time, with $O(n \lg n)$ worst-case delay per letter. Importantly, both approaches [20] and [23] solve the problem under additional constraints on the space used by the algorithm: *compact space* for the former and zero-entropy *compressed space* for the latter.

We here show that, without the compact space constraint, we can efficiently compute LRS in

near-real-time as a by-product of maintaining ST. Observe that under Weiner’s approach described in Section 2.1, the problem becomes the following.

LONGESTREPEATINGPREFIX

Input: suffix tree $\text{ST}(T[i + 1..])$ and letter $T[i]$.

Output: the longest repeating prefix of $T[i..]$.

We solve LONGESTREPEATINGPREFIX by revisiting the methodology of Amir et al. [2]. They reduce the problem to INSERTIONPOINT, since the string label of the insertion point is the longest prefix of $T[i..]$ that already appears in $T[i + 1..]$. Thus, updating $\text{ST}(\overleftarrow{T})$ using Weiner’s algorithm allows us to compute LRS. When processing letter $T[i]$, we query LONGESTREPEATINGPREFIX on $\text{ST}(\overleftarrow{T[1..i - 1]})$ with letter $T[i]$. The answer is $\text{LRS}[i]$.

Theorem 3.1. The longest repeating suffix array LRS of a string $T[1..n]$ can be computed online in $O(t_{\text{SU}})$ worst-case time per letter.

4 Lempel–Ziv factorization (LZ77)

The LZ77 factorization of a string T is defined as follows: a factorization $T = F_1 \cdots F_z$ it is the LZ77 factorization of T if each next factor F_x , for $x \in [1..z]$, is either the first occurrence of a letter or the longest prefix of $F_x \cdots F_z$ that occurs at least twice in $F_1 \cdots F_x$. The factorization can be written like a macro scheme [25], i.e., by a list storing either plain letters or pairs of referred positions and lengths, where a referred position is a previous text position from where the letters of the respective factor can be copied.

While practical compressors based the LZ77 [30] factorization such as gzip and 7zip work with a sliding window over the input text, the original LZ77 factorization considers the entire text as the search buffer. For the online setting, Gusfield’s textbook [13, APL 16] presents an algorithm that runs in $O(\log \sigma)$ amortized time per letter. The main idea is to maintain the suffix tree of the text read so far using Ukkonen’s algorithm [27], and simultaneously compute each next factor by a top-down traversal of the tree. Since we can easily extract the LZ77 factorization from the LRS or LPF arrays, the algorithm of Section 3 can be also used to compute it. Direct online algorithms for computing the LZ77 factorization have also been proposed, additionally focusing on space saving. In this line of research, Starikovskaya [24] proposed an algorithm in $O(\log^2 n)$ amortized time per letter by returning to Ukkonen’s suffix tree construction algorithm. Subsequently, Yamamoto et al. [29] obtained $O(\log n)$ amortized time per letter by constructing the directed acyclic word graph [3], and finally Policriti and Prezesa [21] achieved the same time bounds with a dynamic FM-index in zero-order entropy compressed space.

Here we diverge from this line of research by allowing $O(n)$ words of space but focusing on the worst-case time per letter. Using the near-real-time computation of LRS (Section 3), it is straightforward to compute the LZ77 factorization within the same time bounds.

Online computation of LZ77 factorization amounts to reporting if the current position i extends the current factor or starts a new one. In the latter case, the previous factor ends at position $i - 1$ and i is the first position of a new factor. We can determine this by comparing $i - \text{LRS}[i]$ with the beginning s_x of the current factor F_x . If $i - \text{LRS}[i] + 1 \leq s_x$, then i extends F_x , otherwise F_x ends at $i - 1$ and i starts a new factor F_{x+1} . We observe that $\text{LRS}[i] = 0$ if and only if $T[i]$ is the leftmost occurrence of a letter.

Theorem 4.1. The LZ77 factorization of a string $T[1..n]$ can be computed online in $O(t_{\text{SU}})$ worst-case time per letter.

References

- [1] A. Amir, T. Kopelowitz, M. Lewenstein, and N. Lewenstein. Towards real-time suffix tree construction. In *Proc. SPIRE*, volume 3772 of *LNCS*, pages 67–78, 2005.

- [2] A. Amir, G. M. Landau, and E. Ukkonen. Online timestamped text indexing. *Inf. Process. Lett.*, 82(5):253–259, 2002.
- [3] A. Blumer, J. Blumer, D. Haussler, A. Ehrenfeucht, M. T. Chen, and J. I. Seiferas. The smallest automaton recognizing the subwords of a text. *Theor. Comput. Sci.*, 40:31–55, 1985.
- [4] D. Breslauer and G. F. Italiano. Near real-time suffix tree construction via the fringe marked ancestor problem. *J. Discrete Algorithms*, 18:32–48, 2013.
- [5] M. Burrows and D. J. Wheeler. A block sorting lossless data compression algorithm. Technical Report 124, Digital Equipment Corporation, Palo Alto, California, 1994.
- [6] R. Cole and R. Hariharan. Dynamic LCA queries on trees. *SIAM J. Comput.*, 34(4):894–923, 2005.
- [7] R. Cole, T. Kopelowitz, and M. Lewenstein. Suffix trays and suffix trists: Structures for faster text indexing. *Algorithmica*, 72(2):450–466, 2015.
- [8] P. F. Dietz and R. Raman. Persistence, amortization and randomization. In *Proc. SODA*, pages 78–88, 1991.
- [9] J. Fischer and P. Gawrychowski. Alphabet-dependent string searching with wexponential search trees. In *Proc. CPM*, volume 9133 of *LNCS*, pages 160–171, 2015.
- [10] M. L. Fredman and M. E. Saks. The cell probe complexity of dynamic data structures. In *Proc. STOC*, pages 345–354, 1989.
- [11] Z. Galil. Real-time algorithms for string-matching and palindrome recognition. In *Proc. STOC*, pages 161–173, 1976.
- [12] Y. Giora and H. Kaplan. Optimal dynamic vertical ray shooting in rectilinear planar subdivisions. *ACM Trans. Algorithms*, 5(3), July 2009.
- [13] D. Gusfield. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.
- [14] A. Hartman and M. Rodeh. Optimal parsing of strings. In *Combinatorial Algorithms on Words*, pages 155–167, Berlin, Heidelberg, 1985.
- [15] T. Kopelowitz. On-line indexing for general alphabets via predecessor queries on subsets of an ordered list. In *Proc. FOCS*, pages 283–292, 2012.
- [16] G. Kucherov and Y. Nekrich. Full-fledged real-time indexing for constant size alphabets. *Algorithmica*, 79(2):387–400, 2017.
- [17] C. W. Mortensen. Fully dynamic orthogonal range reporting on RAM. *SIAM J. Comput.*, 35(6):1494–1525, 2006.
- [18] G. Navarro and Y. Nekrich. Optimal dynamic sequence representations. *SIAM J. Comput.*, 43(5):1781–1806, 2014.
- [19] T. Ohno, Y. Takabatake, T. I, and H. Sakamoto. A faster implementation of online run-length Burrows–Wheeler transform. In *Proc. IWOCA*, volume 10765 of *LNCS*, pages 409–419, 2017.
- [20] D. Okanohara and K. Sadakane. An online algorithm for finding the longest previous factors. In *Proc. ESA*, volume 5193 of *LNCS*, pages 696–707, 2008.
- [21] A. Policriti and N. Prezza. Fast online Lempel–Ziv factorization in compressed space. In *Proc. SPIRE*, volume 9309 of *LNCS*, pages 13–20, 2015.

- [22] A. Policriti and N. Prezza. LZ77 computation based on the run-length encoded BWT. *Algorithmica*, 80(7):1986–2011, 2018.
- [23] N. Prezza and G. Rosone. Faster online computation of the succinct longest previous factor array. In *Proc. CiE*, volume 12098 of *LNCs*, pages 339–352, 2020.
- [24] T. Starikovskaya. Computing lempel–ziv factorization online. In *Proc. MFCS*, volume 7464 of *LNCs*, pages 789–799, 2012.
- [25] J. A. Storer and T. G. Szymanski. Data compression via textural substitution. *J. ACM*, 29(4):928–951, 1982.
- [26] T. Takagi, S. Inenaga, H. Arimura, D. Breslauer, and D. Hendrian. Fully-online suffix tree and directed acyclic word graph construction for multiple texts. *Algorithmica*, 82(5):1346–1377, 2020.
- [27] E. Ukkonen. On-line construction of suffix trees. *Algorithmica*, 14(3):249–260, 1995.
- [28] P. Weiner. Linear pattern matching algorithms. In *Proc. 14th Symposium on Switching and Automata Theory (SWAT)*, pages 1–11, 1973.
- [29] J. Yamamoto, T. I. H. Bannai, S. Inenaga, and M. Takeda. Faster compact on-line lempel-ziv factorization. In *Proc. STACS*, volume 25 of *LIPICs*, pages 675–686, 2014.
- [30] J. Ziv and A. Lempel. A universal algorithm for sequential data compression. *IEEE Trans. Information Theory*, 23(3):337–343, 1977.
- [31] J. Ziv and A. Lempel. Compression of individual sequences via variable-rate coding. *IEEE Trans. Information Theory*, 24(5):530–536, 1978.